

BAM Engine: A vectorized Python framework for the Bottom-up Adaptive Macroeconomics model

Konstantinos Ganitis ¹

¹ Department of Informatics, University of Piraeus, Greece

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 07 July 2026

Published: unpublished

License

Authors of papers retain copyright⁴ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

BAM Engine is an open-source Python framework that implements the Bottom-up Adaptive Macroeconomics (BAM) model of Delli Gatti et al. (2011), a canonical agent-based macroeconomic model from the CATS (Complex Adaptive Trivial Systems) family. The model populates an artificial economy with heterogeneous firms, households, and banks that interact across three markets: labor, credit, and consumption goods. Rather than imposing aggregate behavior through representative-agent equations, the model lets macroeconomic phenomena such as growth, unemployment, inflation, and endogenous business cycles emerge from the decentralized, boundedly rational decisions of many individual agents (Delli Gatti et al., 2005; Tesfatsion, 2006).

BAM Engine is validated against quantitative targets from the source text and benchmarked against four other agent-based modeling frameworks, and it ships three extensions that demonstrate how the model can be varied and extended.

Statement of need

Agent-based macroeconomic models are usually disseminated as systems of equations and prose, frequently without accompanying source code. The BAM model in particular is specified verbally (Delli Gatti et al., 2011), so every researcher who studies it must re-implement it from the text and make independent interpretive choices along the way. Results are therefore hard to reproduce, compare, or build upon, which is the central obstacle this framework addresses: it provides a single, inspectable, validated reference implementation that others can run, audit, and modify.

BAM Engine's intended users are researchers, educators, and students in agent-based computational economics who need a fast and modifiable reference implementation, in order to reproduce the book's results, to experiment with parameters and model variants, and to run the large ensembles of simulations that quantitative analysis demands.

State of the field

General-purpose ABM frameworks ease the engineering burden but impose a trade-off. Object-oriented toolkits such as Mesa in Python (Hoeven et al., 2025; Kazil et al., 2020), NetLogo (Wilensky, 1999), and Repast (North et al., 2013) represent each agent as an object and dispatch behavior agent by agent. That design is flexible but scales poorly to the thousands of long runs needed for calibration, sensitivity, and robustness studies; the benchmark reported below measures idiomatic per-agent ports of the same model running an order of magnitude slower than BAM Engine at moderate scale. The mesa-frames project (Amer & Mesa Team and Contributors, 2024) independently diagnoses the same per-agent overhead and retrofits

columnar storage onto Mesa, but it remains a general-purpose, early-stage library with no economic primitives. Domain-specific macro models such as EURACE (Deissenberg et al., 2008) achieve scale, yet they are monolithic research artifacts (C with MPI) that expose no reusable API or extension system.

For the baseline BAM model specifically, existing implementations are limited: a NetLogo port (per-agent objects, not designed for large-scale experiments) and the Julia-based ABCredit engine wrapped by R-MABM (Brusatin et al., 2024), a reinforcement-learning variant of the CATS credit model that requires a Julia runtime. To the author's knowledge, BAM Engine is the first vectorized, pure-Python framework for the baseline BAM model that combines a reusable, extensible architecture with systematic validation against the original text.

To ground this comparison in measurement, the repository ships a cross-framework benchmark (the comparison/ package) that runs the identical baseline BAM model on five frameworks: BAM Engine, Mesa (Kazil et al., 2020), mesa-frames (Amer & Mesa Team and Contributors, 2024), Agents.jl (Datseris et al., 2022), and a pre-existing third-party NetLogo implementation (Platas-López & Guerra-Hernández, 2020). Before any timing is counted, each port must pass a behavioral-equivalence gate: twenty seeds compared against the BAM Engine reference on six metrics computed after a 500-period burn-in (mean unemployment and inflation levels, firm-size skewness, and the Phillips, Okun, and Beveridge correlations), so the comparison measures the same model rather than five similar ones. The NetLogo implementation participates as a non-blocking cross-language reference: a separate third-party port, taken as is, that deviates on several of these metrics. All timings were collected on a single machine (Apple M4 Pro); jobs are single-threaded and run serially, and the result snapshots, environment captures, and port-fairness methodology are committed with the repository.

At 1,000 firms (6,100 agents in total), BAM Engine simulates a period in about 1.5 ms: roughly 11x faster than the idiomatic Mesa port, 13x faster than mesa-frames, and 1.4x faster than the compiled Agents.jl port. At 20,000 firms (122,000 agents), BAM Engine with the optional Numba kernel runs 3.7x faster than Agents.jl, while Mesa and mesa-frames do not complete within budget beyond 5,000 firms. The results split by execution model rather than by language: the ports that reach the population sizes ensemble studies require are those that escape the interpreted per-agent loop, either by compiling it outright (Agents.jl) or by vectorizing the model and compiling the loops that resist vectorization (BAM Engine). BAM Engine is the fastest of the group from 500 firms upward (Figure 1).

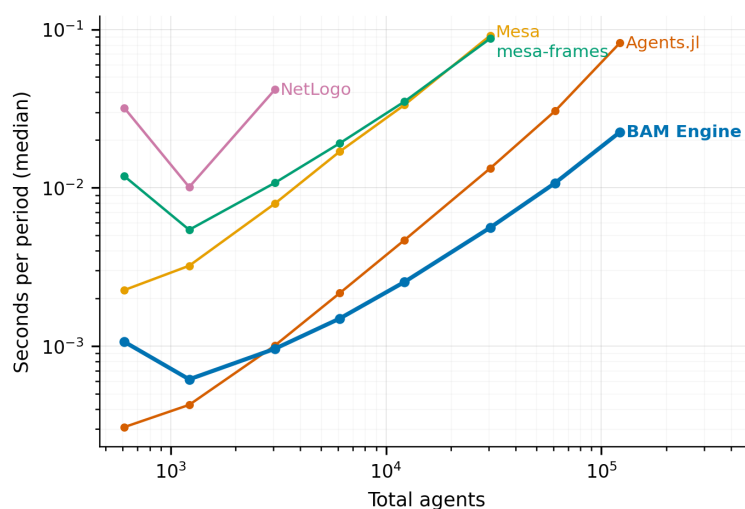


Figure 1: Steady-state per-period wall time against total agent count, log-log. Mesa, mesa-frames, and NetLogo hit the per-job time budget at smaller populations than BAM Engine and the compiled Agents.jl port (Apple M4 Pro).

71 Software design

72 The framework is built on an Entity-Component-System (ECS) architecture that follows
73 data-oriented design principles. Agent state is not held in per-agent objects; instead, each
74 behavioral facet (a “role”) is a structure of parallel NumPy arrays (Harris et al., 2020), and
75 each economic process (an “event”) is a stateless system that transforms those arrays in place.
76 Events run in an explicit, YAML-configurable pipeline organized into the model’s economic
77 phases. This layout replaces per-agent Python loops with vectorized array operations, and it lets
78 researchers reorder, reconfigure, and extend the model (adding new roles, events, relationships,
79 and market mechanisms) without modifying the engine core. Three extensions ship with the
80 framework and demonstrate the mechanism: R&D-driven productivity growth (Russo et al.,
81 2007), buffer-stock consumption (Carroll, 1997), and profit taxation. The package depends
82 only on NumPy and PyYAML at runtime, is fully type-annotated, tested under continuous
83 integration across operating systems and Python versions, and documented with a tutorial-style
84 user guide and runnable examples.

85 Two design trade-offs are worth stating. First, vectorization is applied selectively: the labor
86 and credit markets use batched array matching with a sparse per-row sampler, while the
87 goods market intentionally remains a sequential loop because its purchase-by-purchase state
88 updates make vectorized approximations behaviorally incorrect; an optional Numba kernel (`pip`
89 `install bamengine[fast]`) compiles that loop instead, producing bit-identical results roughly
90 twice as fast at scale. Second, the event pipeline is ordered explicitly in YAML rather than
91 derived from a dependency graph: economic causality in the BAM model is an assumption to
92 be inspected and varied, not an implementation detail to be inferred.

93 Research impact statement

94 BAM Engine is validated against quantitative targets extracted from Delli Gatti et al. (2011)
95 using a two-layer scheme that pairs discrete PASS/WARN/FAIL status checks with continuous
96 scores in the range 0 to 1, the latter feeding a multi-phase calibration pipeline. Across 1,000
97 random seeds of 1,000 periods each, the baseline scenario passes for 98.8% of seeds, the
98 R&D growth scenario for 97.2%, and the buffer-stock scenario for 98.8% (results committed
99 with release v0.10.1). The implementation reproduces canonical stylized facts of the model,
100 including a strong Okun’s-law relationship (mean correlation of about -0.87), Phillips and
101 Beveridge curves, right-skewed firm-size distributions, and financial fragility that co-moves
102 with output in a Minsky-type manner. A robustness suite provides multi-seed internal-validity
103 checks, univariate sensitivity analysis, and structural experiments.

104 The implementation has also surfaced structural properties of the BAM model that the source
105 text does not discuss. Two are documented as hypotheses: a labor-market “quantization
106 trap,” in which integer rounding of labor demand produces a one-way ratchet that suppresses
107 production-driven layoffs, and a credit-market “Kalecki trap,” in which the profit identity drives
108 firm net worth toward a steady-state attractor that nearly eliminates borrowing. Such properties
109 become visible only through mechanism-level inspection of an open, running implementation,
110 and the extension system provides the controlled variation needed to probe them further.

111 AI usage disclosure

112 Anthropic Claude models (Claude Sonnet and Opus families, versions 3.7 through 4.8), used
113 through the Claude Code development tool (2025-2026), assisted with code generation,
114 refactoring, test authoring and documentation writing under the author’s direction and
115 continuous supervision. The author made all core design and architectural decisions (the
116 entity-component-system layout, the event pipeline, the validation methodology and targets,
117 the benchmark design and its behavioral-equivalence gate), reviewed and validated every

change against the source text and the committed test and validation suites, and takes full responsibility for the correctness, originality, and licensing of the software.

Acknowledgements

BAM Engine was developed as part of MSc thesis research at the University of Piraeus.

References

- Amer, A., & Mesa Team and Contributors. (2024). *mesa-frames: Extension of Mesa for performance and scalability* (Version 0.1.0a0). Project Mesa. <https://github.com/projectmesa/mesa-frames>
- Brusatin, S., Padoan, T., Coletta, A., Delli Gatti, D., & Glielmo, A. (2024). Simulating the economic impact of rationality through reinforcement learning and agent-based modelling. *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF '24)*, 159–167. <https://doi.org/10.1145/3677052.3698621>
- Carroll, C. D. (1997). Buffer-stock saving and the life cycle/permanent income hypothesis. *The Quarterly Journal of Economics*, 112(1), 1–55. <https://doi.org/10.1162/003355397555109>
- Datseris, G., Vahdati, A. R., & DuBois, T. C. (2022). Agents.jl: A performant and feature-full agent-based modeling software of minimal code complexity. *SIMULATION*, 100(10), 1019–1031. <https://doi.org/10.1177/00375497211068820>
- Deissenberg, C., Hoog, S. van der, & Dawid, H. (2008). EURACE: A massively parallel agent-based model of the european economy. *Applied Mathematics and Computation*, 204(2), 541–552. <https://doi.org/10.1016/j.amc.2008.05.116>
- Delli Gatti, D., Desiderio, S., Gaffeo, E., Cirillo, P., & Gallegati, M. (2011). *Macroeconomics from the bottom-up*. Springer. <https://doi.org/10.1007/978-88-470-1971-3>
- Delli Gatti, D., Di Guilmi, C., Gaffeo, E., Giulioni, G., Gallegati, M., & Palestini, A. (2005). A new approach to business fluctuations: Heterogeneous interacting agents, scaling laws and financial fragility. *Journal of Economic Behavior & Organization*, 56(4), 489–512. <https://doi.org/10.1016/j.jebo.2003.10.012>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hoeven, E. ter, Kwakkel, J., Hess, V., Pike, T., Wang, B., rht, & Kazil, J. (2025). Mesa 3: Agent-based modeling with Python in 2025. *Journal of Open Source Software*, 10(107), 7668. <https://doi.org/10.21105/joss.07668>
- Kazil, J., Masad, D., & Crooks, A. (2020). Utilizing Python for agent-based modeling: The Mesa framework. In R. Thomson, H. Bisgin, C. Dancy, A. Hyder, & M. Hussain (Eds.), *Social, cultural, and behavioral modeling (SBP-BRiMS 2020)* (Vol. 12268, pp. 308–317). Springer. https://doi.org/10.1007/978-3-030-61255-9_30
- North, M. J., Collier, N. T., Ozik, J., Tatara, E. R., Macal, C. M., Bragen, M., & Sydelko, P. (2013). Complex adaptive systems modeling with Repast Symphony. *Complex Adaptive Systems Modeling*, 1(1), 3. <https://doi.org/10.1186/2194-3206-1-3>
- Platas-López, A., & Guerra-Hernández, A. (2020). *NetLogo implementation of the BAM model*. CoMSES Computational Model Library. <https://doi.org/10.25937/tfx7-y446>

- 161 Russo, A., Catalano, M., Gaffeo, E., Gallegati, M., & Napoletano, M. (2007). Industrial
162 dynamics, fiscal policy and R&D: Evidence from a computational experiment. *Journal of*
163 *Economic Behavior & Organization*, 64(3–4), 426–447. [https://doi.org/10.1016/j.jebo.](https://doi.org/10.1016/j.jebo.2006.06.016)
164 [2006.06.016](https://doi.org/10.1016/j.jebo.2006.06.016)
- 165 Tesfatsion, L. (2006). Agent-based computational economics: A constructive approach to
166 economic theory. In L. Tesfatsion & K. L. Judd (Eds.), *Handbook of computational*
167 *economics* (Vol. 2, pp. 831–880). Elsevier. [https://doi.org/10.1016/S1574-0021\(05\)](https://doi.org/10.1016/S1574-0021(05)02016-2)
168 [02016-2](https://doi.org/10.1016/S1574-0021(05)02016-2)
- 169 Wilensky, U. (1999). *NetLogo*. Center for Connected Learning and Computer-Based Modeling,
170 Northwestern University. <http://ccl.northwestern.edu/netlogo/>

DRAFT